Lab Handout 2-G

End-to-End Pipeline Simulation with Recorded Interceptor Data

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

---

1 Objective

Stand up a single testbench that drives the complete Module 2 pipeline — `uart_rx` → `byte_fifo` → `pkt_parser` (with Cyclic Redundancy Check (CRC) engines) → `feature_extract` → `feature_bram` — using a recorded Module 1 interceptor capture, and compare every exported feature vector against a Python reference.

2 Prerequisites

- Lab Handouts 2-A through 2-F (full pipeline integrated under `ingest_top`).
- Recorded Module 1 interceptor capture (`.bin`) with at least 1000 frames covering the cooperative-target ping-pong (DATA/ACK from Beacon IDs `0x01` and `0x02`) and at least one button-hold burst of PAUSE traffic from the Threat (Beacon ID `0xFF`).

3 Procedure

3.1 Step 1 — Scenario selection

Pick three test captures from Module 1 and place them in `data/`:

1. `capture_clean.bin` — only Target A initiating DATA, no responder, no PAUSE.
2. `capture_mixed.bin` — Target A DATA + Target B ACK ping-pong with at least one button-hold PAUSE window.
3. `capture_corrupt.bin` — generated from `capture_mixed.bin` by the Python helper in Step 4 below; introduces bit flips in roughly 5% of frames.

3.2 Step 2 — Top-level testbench

Add a simulation source `tb_ingest_top`. Replace its contents with the listing below.

Enable VHDL-2008 in Vivado before adding this file

The testbench uses a VHDL-2008 *external name* alias to read the internal `host_rd_data_int` signal inside `ingest_top`. Vivado defaults each new file to VHDL-93, which rejects external names. Right-click `tb_ingest_top.vhd` in the Sources panel → **Source File Properties** → **File Type** and set it to **VHDL 2008** *before* the first compile. The DUT instance label in the testbench must remain `dut` (line `dut : entity work.ingest_top`); the external name path

`.tb_ingest_top.dut.host_rd_data_int` resolves against that label.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;

entity tb_ingest_top is
end tb_ingest_top;

architecture sim of tb_ingest_top is
    constant CLK_PERIOD   : time   := 10 ns;
    constant BIT_PERIOD   : time   := 8680 ns;
    constant CAPTURE_FILE : string := "../../../../data/capture_mixed.bin";
    constant DUMP_FILE    : string := "../../../../data/dump_mixed.txt";
    constant LOG_FILE     : string := "../../../../data/events_mixed.txt";

    signal clk          : std_logic := '0';
    signal cpu_resetn   : std_logic := '0';
    signal uart_rx_pin  : std_logic := '1';
    signal uart_rxd_out : std_logic;
    signal sw           : std_logic_vector(15 downto 0) := (others => '0');

    signal ca, cb, cc, cd, ce, cf, cg : std_logic;
    signal dp                         : std_logic;
    signal an                         : std_logic_vector(7 downto 0);
    signal led                        : std_logic_vector(15 downto 0);

    type charfile is file of character;

    procedure send_byte (signal s : out std_logic;
                         b : in std_logic_vector(7 downto 0)) is
    begin
        s <= '0'; wait for BIT_PERIOD;
        for i in 0 to 7 loop
            s <= b(i); wait for BIT_PERIOD;
        end loop;
        s <= '1'; wait for BIT_PERIOD;
    end procedure;

    -- VHDL-2008 external name reaches the internal 256-bit BRAM readback
    -- signal inside ingest_top so the testbench can dump the full feature row.
    alias host_rd_data_int is
        << signal .tb_ingest_top.dut.host_rd_data_int :
           std_logic_vector(255 downto 0) >>;
begin
    clk <= not clk after CLK_PERIOD / 2;

    dut : entity work.ingest_top
        port map (CLK100MHZ => clk, CPU_RESETN => cpu_resetn,
```

```
                    uart_rx_pin => uart_rx_pin,
                    UART_RXD_OUT => uart_rxd_out,
                    SW => sw,
                    CA => ca, CB => cb, CC => cc, CD => cd,
                    CE => ce, CF => cf, CG => cg,
                    DP => dp, AN => an, LED => led);

    -- Event log: timestamped frame_valid / frame_reject pulses.
    -- led(1) = frame_valid, led(4) = frame_reject (per LH2-F ingest_top).
    process
        file     fh        : text open write_mode is LOG_FILE;
        variable l         : line;
        variable t_ns      : integer;
        variable prev_fv   : std_logic := '0';
        variable prev_fr   : std_logic := '0';
    begin
        loop
            wait until rising_edge(clk);
            if led(1) = '1' and prev_fv = '0' then
                t_ns := now / 1 ns;
                write(l, string'("VALID @ "));
                write(l, t_ns);
                writeline(fh, l);
            end if;
            if led(4) = '1' and prev_fr = '0' then
                t_ns := now / 1 ns;
                write(l, string'("REJECT @ "));
                write(l, t_ns);
                writeline(fh, l);
            end if;
            prev_fv := led(1);
            prev_fr := led(4);
        end loop;
    end process;

    process
        file     fh   : charfile;
        file     fh_d : text open write_mode is DUMP_FILE;
        variable ch   : character;
        variable b    : std_logic_vector(7 downto 0);
        variable l    : line;
        variable open_status : file_open_status;
    begin
        wait for 200 ns;
        cpu_resetn <= '1';
        wait for 1 us;

        file_open(open_status, fh, CAPTURE_FILE, read_mode);
        assert open_status = open_ok
            report "Could not open " & CAPTURE_FILE severity failure;

        while not endfile(fh) loop
            read(fh, ch);
            b := std_logic_vector(to_unsigned(character'pos(ch), 8));
```

```vhdl
            send_byte(uart_rx_pin, b);
        end loop;
        file_close(fh);

        -- Drain time: allow last frame to traverse parser + extractor + BRAM.
        wait for 500 us;

        -- Walk every row of the BRAM and dump as hex strings. SW(7:0) drives
        -- the row address inside ingest_top; SW(12:8) is unused for the dump
        -- (the testbench reads the full 256-bit row via the external name).
        for row in 0 to 255 loop
            sw(7 downto 0) <= std_logic_vector(to_unsigned(row, 8));
            wait until rising_edge(clk);
            wait until rising_edge(clk);              -- 1-cycle BRAM read latency
            write(l, string'("ROW "));
            hwrite(l, std_logic_vector(to_unsigned(row, 8)));
            write(l, string'(" "));
            hwrite(l, host_rd_data_int);
            writeline(fh_d, l);
        end loop;

        report "tb_ingest_top: dump complete" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 1: `tb_ingest_top.vhd` — file-driven end-to-end harness. Streams every byte of `CAPTURE_FILE` at 115200 baud through `ingest_top`, then walks the Block Random Access Memory (BRAM) read port over all 256 rows (using `SW(7:0)` as the row address) and dumps the full 256-bit row contents (read via VHDL-2008 external name) to `DUMP_FILE`. Requires VHDL-2008 (set in Vivado's Project Settings → General).

3.3 Step 3 — Python reference and corruption helper

Create `ref_pipeline.py` under `Code/Tools/RefPipeline/` (alongside `RawCapture/` from LH2-A). The script: (1) pre-processes `capture_mixed.bin` into `capture_corrupt.bin` with random bit flips; (2) reads any of the three captures, parses frames, accumulates per-Beacon-ID feature vectors using the same rules as `feature_extract.vhd`, and writes them in the same hex-string format as the testbench dump. Drop a one-line `requirements.txt` alongside the script for layout consistency with the other tools (the script itself uses only the Python standard library).

```python
"""Module 2 reference pipeline: produces the same per-row hex dump that
tb_ingest_top.vhd writes to data/dump_*.txt. Diff the two files for a
byte-for-byte regression of the full pipeline."""
import argparse, struct, random, pathlib

CRC8_POLY  = 0x07
CRC16_POLY = 0x1021
```

```python

def crc8(data: bytes) -> int:
    c = 0
    for b in data:
        c ^= b
        for _ in range(8):
            c = ((c << 1) ^ CRC8_POLY) & 0xFF if c & 0x80 else (c << 1) & 0xFF
    return c

def crc16(data: bytes) -> int:
    c = 0xFFFF
    for b in data:
        c ^= b << 8
        for _ in range(8):
            c = ((c << 1) ^ CRC16_POLY) & 0xFFFF if c & 0x8000 else (c << 1) & 0xFFFF
    return c

def corrupt(src: pathlib.Path, dst: pathlib.Path, frac: float = 0.05, seed: int = 1):
    """Flip one random bit in roughly `frac` of frames."""
    rng  = random.Random(seed)
    data = bytearray(src.read_bytes())
    i = 0
    while i + 43 <= len(data):
        if data[i] == 0x7E and data[i + 1] == 0x29:
            if rng.random() < frac:
                off = rng.randrange(2, 42)         # don't touch delimiter or length
                bit = 1 << rng.randrange(8)
                data[i + off] ^= bit
            i += 43
        else:
            i += 1
    dst.write_bytes(data)

def parse(stream: bytes):
    """Yield 43-byte frames whose CRCs both check out."""
    i = 0
    while i + 43 <= len(stream):
        if stream[i] != 0x7E or stream[i + 1] != 0x29:
            i += 1
            continue
        frame = stream[i:i + 43]
        if crc8(frame[1:42]) != frame[42]:
            i += 43
            continue
        embedded = (frame[40] << 8) | frame[41]
        if crc16(frame[10:40]) != embedded:
            i += 43
            continue
        yield frame
        i += 43
```

```python

def s8(b: int)  -> int: return b - 256 if b >= 128 else b
def s16(v: int) -> int: return v - 0x10000 if v >= 0x8000 else v

def process(stream: bytes):
    rows = {b: bytearray(32) for b in range(256)}
    state = {}                                      # beacon -> (ts_last, rssi_last, count)
    for f in parse(stream):
        ts   = struct.unpack("<I", f[2:6])[0]
        rssi = s8(f[6])
        snr  = s8(f[7])
        radio    = f[10:42]
        beacon   = radio[2]
        mod_code = radio[6]
        sf       = radio[7]
        pkt_type = mod_code & 0x03
        mod_kind = mod_code >> 2
        old_ts, old_rssi, old_count = state.get(beacon, (0, 0, 0))
        new_count = old_count + 1
        inter_arr = (ts - old_ts) & 0xFFFFFFFF
        rssi_delt = (rssi - old_rssi) & 0xFFFFFFFF
        priority  = ((rssi << 2) + snr + new_count) & 0xFFFFFFFF
        tgt = radio[13] if pkt_type == 2 else 0    # LH1-G buf[13]
        dur = radio[14] if pkt_type == 2 else 0    # LH1-G buf[14]
        v = bytearray(32)
        v[0]   = beacon
        v[1]   = mod_kind
        v[2]   = sf
        v[3]   = pkt_type
        struct.pack_into("<h", v, 4,  rssi)
        struct.pack_into("<h", v, 6,  snr)
        struct.pack_into("<I", v, 8,  new_count)
        struct.pack_into("<I", v, 12, ts)
        struct.pack_into("<I", v, 16, inter_arr)
        struct.pack_into("<I", v, 20, rssi_delt)
        struct.pack_into("<I", v, 24, priority)
        v[28] = tgt
        v[29] = dur
        rows[beacon] = v
        state[beacon] = (ts, rssi, new_count)
    return rows

def dump(rows, path: pathlib.Path):
    with path.open("w") as f:
        for r in range(256):
            # Hardware dump prints byte 31 first (MSB of the packed 256-bit vector
            # is byte 31). Match that ordering exactly.
            hex_be = rows[r][::-1].hex().upper()
            f.write(f"ROW {r:02X} {hex_be}\n")

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument("--capture", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--corrupt-from")
    ap.add_argument("--seed", type=int, default=1)
    args = ap.parse_args()
    cap = pathlib.Path(args.capture)
    if args.corrupt_from:
        corrupt(pathlib.Path(args.corrupt_from), cap, seed=args.seed)
    rows = process(cap.read_bytes())
    dump(rows, pathlib.Path(args.out))
    print(f"wrote {args.out}")
```

Listing 2: `Code/Tools/RefPipeline/ref_pipeline.py` — byte-for-byte reference for the Module 2 hardware pipeline.

3.4 Step 4 — Generate captures and references

Run the helper from a PowerShell terminal at the repo root. The corrupt-from invocation produces `capture_corrupt.bin` *and* its reference dump in one shot:

```
python "Code/Tools/RefPipeline/ref_pipeline.py" --corrupt-from "Code/ingest_top/data/capture_mixed.bin" --capture "Code/ingest_top/data/capture_corrupt.bin" --out "Code/ingest_top/data/ref_corrupt.txt"
python "Code/Tools/RefPipeline/ref_pipeline.py" --capture "Code/ingest_top/data/capture_clean.bin" --out "Code/ingest_top/data/ref_clean.txt"
python "Code/Tools/RefPipeline/ref_pipeline.py" --capture "Code/ingest_top/data/capture_mixed.bin" --out "Code/ingest_top/data/ref_mixed.txt"
```

Listing 3: Shell commands (PowerShell-friendly single lines) to produce the corrupted capture and all three reference dumps.

3.5 Step 5 — Run the three simulations and diff

For each scenario, edit the `CAPTURE_FILE` and `DUMP_FILE` constants at the top of `tb_ingest_top.vhd`, set `tb_ingest_top` as the simulation top (right-click in Sources → Simulation Sources → **Set as Top**), and launch a behavioral simulation. The default `1us` runtime is far short of the ∼5.5 simulated seconds a 1500-frame capture requires (each byte takes 87 μs at 115200 baud). Either bump `xsim.simulate.runtime` to `6sec` in *Project Settings* → *Simulation* before launching, or after the launch finishes type `run 6 sec` (once) in the Tcl console. Wall-clock cost is roughly 4–6 minutes per scenario on a typical laptop. Then diff the dump against the reference:

```
diff Code/ingest_top/data/dump_clean.txt   Code/ingest_top/data/ref_clean.txt
diff Code/ingest_top/data/dump_mixed.txt   Code/ingest_top/data/ref_mixed.txt
diff Code/ingest_top/data/dump_corrupt.txt Code/ingest_top/data/ref_corrupt.txt
```

Listing 4: Compare each simulation dump against its Python reference. PowerShell users: substitute `Compare-Object` or use Git Bash / WSL for `diff`.

All three diffs must be empty.

3.6 Step 6 — Pass criteria

- **Clean single-station**: `diff` produces no output. Row `0x01` has `pkt_type = 0x00`, frame count > 0; all other rows are zero.
- **Mixed with PAUSE**: `diff` produces no output. Rows `0x01`, `0x02`, `0xFF` occupied; `pkt_type` = `0x00`, `0x01`, `0x02` respectively.
- **Corrupted**: `diff` produces no output. The number of `REJECT` entries in `events_mixed.txt` equals the number of bit-flipped frames produced by the helper script (the seeded RNG makes this exact and reproducible).

[Figure placeholder]

`LH2G_full_pipeline.png`

Full Module 2 pipeline under `tb_ingest_top`, showing the file-driven stimulus source, `frame_valid`/`frame_reject` probes, and the BRAM dump path.

Figure 1: Full Module 2 pipeline under `tb_ingest_top`, showing the file-driven stimulus source, `frame_valid`/`frame_reject` probes, and the BRAM dump path.

Simulation runtime trap

A 1000-frame capture at 115200 baud is roughly 4 s of simulated wall-clock time. Full-waveform logging across that span is slow and disk-hungry. Disable wave logging for the regression run and re-enable it only for the first 10 frames or for a specific `REJECT` event when investigating a failure.

4 Verification Checklist

- ☐ All three captures exist in `data/`; `capture_corrupt.bin` produced by the seeded Python helper.
- ☐ All three reference dumps exist in `data/` (`ref_clean.txt`, `ref_mixed.txt`, `ref_corrupt.txt`).
- ☐ All three simulation dumps match their reference (zero `diff` output).
- ☐ `REJECT` count in `events_mixed.txt` for the corrupt run equals the number of bit-flipped frames from the helper.

Lab Handout 2-G Deliverable

1. `tb_ingest_top.vhd` (in `ingest_top.srcs/sim_1/new/`) and `Code/Tools/RefPipeline/ref_pipeline.py`.
2. All three diff results (each empty file is acceptable; redirect `diff`'s exit code to a log).
3. Waveform screenshot of one valid frame and one rejected frame from the corrupt run.

Glossary

BRAM	Block Random Access Memory	4, 8
CRC	Cyclic Redundancy Check	1
FPGA	Field-Programmable Gate Array	1
RTL	Register-Transfer Level	1